IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

ETAPA B · CONSTRUIR · SPRINT 7 DE 17

Apuntes del Sprint 7 · Modelos de regresión y clasificación

ANTES DE EMPEZAR

este sprint usa ideas de los Sprints 1 y 6 (matriz de confusión, sobreajuste, baseline, pipeline, validación cruzada). Si alguna te suena a chino, vuelve a esos apuntes.

Las diapositivas del Sprint 7 recorren **siete modelos y muchas métricas** en pocas páginas. Estos apuntes **explican la idea de cada modelo con un ejemplo pequeño que puedes comprobar a mano**, sin fórmulas intimidantes. Léelos antes de cada cuaderno.

0. De qué va el sprint (en 1 minuto)

Hay **dos grandes tipos de problema supervisado** (ya lo viste en el Sprint 1):

	Pregunta	Salida	Ejemplo TurisData	Modelos del sprint
Regresión	¿Cuánto?	Un **número**	¿Cuánto debería costar una noche?	Lineal, polinómica, Ridge, Lasso
Clasificación	¿De qué tipo? ¿Sí o no?	Una **categoría** o probabilidad	¿Se cancelará esta reserva?	Logística, k-NN, Naive Bayes

El reto: dos herramientas para el gerente: (1) una **tarifa de referencia** (regresión) y (2) un **aviso de riesgo de cancelación** para decidir cuándo hacer *overbooking* (clasificación), **traduciendo las métricas a euros**.

PARTE 1 · Regresión: predecir un número

1. La idea: una suma ponderada

La regresión lineal predice el precio como **un precio base más «extras»**:

```
precio ≈ base + (efecto suite)·es_suite + (efecto temporada alta)·temporada_alta
```

Cada coeficiente dice cuántos euros cambia el precio al subir esa variable en una unidad. Ejemplo con valores inventados: `base = 80`, `suite = +45`, `temporada_alta = +22`:

Habitación	Temporada	Cálculo	Precio previsto
Estándar	baja	80	**80 €**
Suite	baja	80 + 45	**125 €**
Estándar	alta	80 + 22	**102 €**
Suite	alta	80 + 45 + 22	**147 €**

Se lee: *«una suite cuesta 45 € más que una estándar»* y *«la temporada alta añade 22 €»*. **Eso es lo que hace atractiva la regresión lineal: se puede explicar al cliente.**

Variables categóricas: `drop_first=True`

Una categoría como `tipo_habitacion` (estándar / suite) se convierte en columnas 0/1. Con `drop_first=True` **se quita una categoría** (estándar) que hace de **referencia**: el modelo solo tiene la columna `es_suite`. Los coeficientes se leen **frente**

a la referencia.

```
modelo_lin = LinearRegression().fit(X_train, y_train)
coefs = pd.Series(modelo_lin.coef_, index=X.columns)       # un coeficiente por columna
```

2. Revisar el modelo: residuos

Residuo = real − predicho. Si una suite real costó 130 € y el modelo dijo 125 €, el residuo es **+5**.

- Un modelo bien planteado deja **residuos sin patrón**, repartidos alrededor de 0.
- **Curvas o un embudo** en los residuos significan que **falta algo**: interacciones, transformar el objetivo (por ejemplo, el logaritmo)…

Interacciones

El extra de la temporada alta **no tiene por qué ser igual** para una estándar y para una suite. Quizá en estándar sube 22 € y en suite 35 €. Esa diferencia es una **interacción** (una variable que «cambia el efecto de otra»). Se mide viendo **cuánto baja el error (MAE) al añadirla**: `mejora_mae = mae_test − mae_con_interaccion`.

3. Polinomios: captar curvas… y su peligro

El precio **sube en invierno y verano y baja en primavera y otoño**: es una **curva** respecto al mes. Una recta no la capta. Un **polinomio de grado 2** (añadir mes²) sí puede doblarse una vez; con grado más alto, más curvas.

```
make_pipeline(PolynomialFeatures(2), LinearRegression())
```

Opción	Idea	Veredicto
Polinomio de grado alto	Imita la curva, pero es **frágil** y difícil de explicar	Riesgo de sobreajuste
`temporada_alta` (variable pensada)	**Simple, explicable**	Suele ganar

> **Pensar bien las variables (feature engineering) casi siempre supera a añadir complejidad al modelo.** Una variable que dice «es temporada alta» vale más que un polinomio de grado 8.

4. Muchas variables, pocos datos: el problema del sobreajuste

Si partes de **20 variables** y añades **todas sus combinaciones de grado 2**, obtienes 20 + 210 = **230 columnas**. Con solo **120 filas** hay más columnas que datos: el modelo **memoriza**.

Síntoma: **una brecha grande entre el R^2 de entrenamiento y el de prueba**.

```
brecha_ols = ols.score(X_train, y_train) - ols.score(X_test, y_test)
```

Ejemplo: R^2 entrenamiento = 0,99 y R^2 prueba = 0,40 → brecha de **0,59**: sobreajuste claro. **La solución: penalizar los coeficientes grandes** (regularización).

5. Ridge, Lasso y ElasticNet

La **regularización** añade al error una **«multa» por cada coeficiente grande**. Así el modelo prefiere explicar con pocos pesos

moderados. La fuerza de la multa se llama **`alpha`**.

	Ridge (L2)	Lasso (L1)	ElasticNet
Penaliza	`alpha · Σ coef²`	`alpha · Σ abs(coef)`	Mezcla de ambas
Efecto	**Encoge todos** los coeficientes; reparte el peso entre variables parecidas	**Lleva a cero** las variables poco útiles	Útil con variables **muy correlacionadas**
¿Deja alguno en cero?	Rara vez	**Sí**	A veces
Para qué	Estabilizar	**Seleccionar variables**	Mixto

Analogía: imagina que cada coeficiente es un empleado y la penalización es un **impuesto por sueldo**. Con Ridge, **todos cobran menos**. Con Lasso, **a los que no aportan, los despides** (coeficiente = 0) y a los demás les bajas el sueldo.

Lasso gana cuando pocas variables lo explican casi todo (nuestro caso) y es fácil de explicar al cliente: *«las 5 cosas que más influyen en el precio»*.

Elegir `alpha`

- `RidgeCV` y `LassoCV` **prueban una lista de `alpha` con validación cruzada** dentro del entrenamiento; **el test no se toca hasta el final**.
- **Escala antes de penalizar**: si una variable va de 0 a 10.000 y otra de 0 a 1, la primera tendrá un coeficiente diminuto y la multa no la afecta; la otra sí. Escalar iguala el terreno.
- **Con muchos datos, regularizar importa menos**: el mini-reto muestra la curva del R^2 en prueba con 60, 120, 300 y 1.000 filas: con 1.000 la diferencia se reduce mucho (R^2 en prueba de unos 0,84 frente a 0,93).

PARTE 2 · Clasificación: la regresión logística

6. De una suma ponderada a una probabilidad

Misma idea que la regresión lineal, pero la salida debe ser una **probabilidad entre 0 y 1**. La **sigmoide** «comprime» cualquier número a ese rango:

```
probabilidad = 1 / (1 + e^(−z))     donde z = suma ponderada
```

z	−2	0	1	2
Probabilidad	0,12	**0,50**	0,73	0,88

- `z` muy negativo → casi 0 (no cancela). `z = 0` → 50 %. `z` muy positivo → casi 1 (cancela).

```
logit = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
logit.fit(X_train, y_train)
proba_test = logit.predict_proba(X_test)[:, 1]      # probabilidad de cancelar
```

- `predict_proba` da la **probabilidad**.
- `predict` **aplica un umbral** (0,5 por defecto) y devuelve 0/1.
- **Todo el preprocesado va dentro del `Pipeline`** (como en el Sprint 6).

7. Interpretar con *odds ratios*

Odds = *probabilidad de que ocurra / probabilidad de que no ocurra*. Es como se expresan las apuestas.

Probabilidad de cancelar	Odds
50 %	0,5 / 0,5 = **1** (igual de probable que no)
75 %	0,75 / 0,25 = **3** (3 contra 1 a favor de cancelar)
20 %	0,2 / 0,8 = **0,25**

`exp(coeficiente)` = ***odds ratio***: **por cuánto se multiplican los odds** cuando esa variable sube una unidad.

Coeficiente	Odds ratio = e^coef	Lectura
+0,69	**2,0**	Los odds de cancelar **se duplican**
0	1,0	Sin efecto
−0,50	**0,61**	Los odds **bajan** (≈ 39 % menos)

> 1 aumenta el riesgo; < 1 lo reduce.

```
odds = pd.Series(np.exp(coefs), index=nombres)
```

> **Cuidado con la causalidad.** Que `tarifa_no_reembolsable` tenga un odds ratio < 1 **no significa que cause menos cancelaciones**: puede ser que **quien la elige ya sea un cliente más seguro** de que viajará. Es la misma lección de «correlación no es causalidad» del Sprint 5.

8. El umbral y la matriz de confusión

	Predicho: se mantiene	Predicho: cancela
Real: se mantiene	Verdadero negativo (VN)	**Falsa alarma (FP)**
Real: cancela	**Cancelación no prevista (FN)**	Verdadero positivo (VP)

El modelo da una **probabilidad**; **nosotros decidimos desde qué valor hablamos de «riesgo»** (el umbral). Bajar el umbral hace que **avisemos más**: **sube el recall** (detectamos más cancelaciones) y **baja la precisión** (más avisos son falsos).

9. Elegir el umbral según el coste

Los errores **no cuestan lo mismo**:

- **FN (cancelación no prevista):** habitación vacía que podíamos haber vendido.
- **FP (falsa alarma):** molestamos a un cliente que iba a venir, o hacemos *overbooking* sin necesidad.

Si **un FN cuesta el triple que un FP** (FN = 3, FP = 1), conviene ser más «alarmista».

Ejemplo numérico completo

10 reservas con la probabilidad que da el modelo y lo que pasó de verdad (4 se cancelaron):

Prob.	0,90	0,80	0,70	0,60	0,50	0,40	0,30	0,20	0,10	0,05
¿Canceló?	sí	sí	no	sí	no	sí	no	no	no	no

Umbral	Avisos	VP	FP	FN	Recall	Precisión	Coste (3·FN + FP)
0,70	3	2	1	2	50 %	67 %	3·2 + 1 = **7**
0,50	5	3	2	1	75 %	60 %	3·1 + 2 = **5**
0,40	6	4	2	0	**100 %**	67 %	3·0 + 2 = **2** ← el mejor
0,30	7	4	3	0	100 %	57 %	0 + 3 = **3**
0,20	8	4	4	0	100 %	50 %	0 + 4 = **4**

(«Aviso» = probabilidad **igual o mayor** que el umbral.) El mejor umbral **no es 0,5**: es **0,40**.

```
umbrales = np.arange(0.05, 0.96, 0.05)
costes = [coste_total(y_train, proba_cv, u, 3, 1) for u in umbrales]      # FN = 3, FP = 1
mejor_umbral = umbrales[int(np.argmin(costes))]                    # el de menor coste
```

> **El gerente no quiere oír «AUC»: quiere euros.** La recomendación al cliente debe incluir al menos **una cifra** («con este umbral evitamos X habitaciones vacías por cada Y avisos de más»). Importante: el umbral se elige **con datos de entrenamiento/validación**, y se **evalúa** en test.

PARTE 3 · Datos desbalanceados, ROC/PR y modelos sencillos

10. La paradoja de la exactitud

Si solo se cancela el **5 %** de las reservas, un modelo «tonto» que **nunca avisa** acierta el **95 %**… y tiene **recall 0**. Ya lo viste en el Sprint 1.

```
tonto = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
```

Siempre se compara con él. **Tres remedios** para clases raras:

Remedio	Qué hace
Pesos por clase (`class_weight="balanced"`)	Cada error en la clase rara **cuenta más**. Con 1.000 reservas y 50 canceladas: peso de la clase mayoritaria 1000/(2·950) ≈ **0,53**; de la rara 1000/(2·50) = **10**
Submuestreo	Se descartan reservas de la clase mayoritaria para equilibrar
Ajustar el umbral	Bajarlo para detectar más casos

> **Remuestrear solo el entrenamiento.** El test se deja **como es en la realidad** (con su 5 %): si lo equilibras, tu evaluación mentirá.

11. ROC frente a PR

	Curva ROC y AUC	Curva PR y AP
Eje	**TPR** (recall) frente a **FPR** (falsas alarmas sobre los negativos)	**Precisión** frente a **recall**
Referencia	La **diagonal** = modelo inútil (AUC 0,5)	La **prevalencia** (proporción de positivos)
Mejor cuando	Clases equilibradas	**Clases raras** (más honesta)

Por qué la ROC puede engañar con clases raras: con 1.000 reservas y 50 canceladas, una tasa de falsas alarmas de solo el 5 % son unos **47 avisos falsos** (5 % de 950); con 40 aciertos de 50, la ROC parece excelente, pero **solo 40 de cada 87 avisos son ciertos** (precisión ≈ 46 %). La PR deja eso a la vista.

La línea de referencia de la PR es la prevalencia: con 5 % de canceladas, un modelo al azar tendría una precisión media de 0,05. Si el tuyo logra 0,30, es **6 veces mejor que el azar** (`veces_mejor_que_azar = ap / prevalencia`).

Umbral con restricción

A veces el negocio dice: *«quiero detectar al menos el 70 % de las cancelaciones»*. Con la curva PR se busca **el umbral más alto que mantenga recall ≥ 0,70**:

```
prec, rec, umbrales = precision_recall_curve(y_train, proba_cv)
umbral_70 = umbrales[rec[:-1] >= 0.70].max()
```

12. k-NN: predecir mirando a los vecinos

Idea: «dime con quién andas y te diré quién eres».

1. Calcula la **distancia** de la reserva nueva a **todas las conocidas**.
2. Quédate con las **k más cercanas** (los vecinos).
3. La clase prevista es **la más frecuente entre ellos** (la probabilidad es la proporción).

Ejemplo: k = 5; de los 5 vecinos, 3 cancelaron y 2 no → predicción «cancela», probabilidad 3/5 = **60 %**.

Rasgos de k-NN:

- **No hay entrenamiento real**: guarda los datos y consulta. Por eso **predecir es lento** con muchos datos.
- **Mide distancias, así que hay que escalar.** Sin escalar, una variable con números grandes domina todo.

Ejemplo: A = (antelación 10 días, 0 niños), B = (40 días, 0 niños), C = (10 días, 3 niños). Sin escalar, la distancia A–B = 30 y A–C = 3: **los tres niños de diferencia casi no cuentan** frente a los 30 días, solo por tener números más pequeños. Escalando, cada variable pesa lo que debe.

Elegir k

	k pequeño (1, 3)	k grande
Problema	**Copia el ruido** de los vecinos: alta varianza, **sobreajuste**	**Promedia demasiado**: alto sesgo, suaviza en exceso

Se elige **probando varios k con validación cruzada** y mirando el AUC: `mejor_k = ks[np.argmax(auc_por_k)]`.

13. Naive Bayes: probabilidades multiplicadas

Se apoya en el **teorema de Bayes** (Sprint 2 y 5):

```
P(cancela | datos)  ∝  P(datos | cancela) · P(cancela)
```

La simplificación «ingenua»: se supone que las variables son **independientes entre sí** dadas la clase. Entonces basta **multiplicar** las probabilidades de cada variable.

Ejemplo completo (con cifras de ejemplo)

Una reserva con tarifa **flexible** y **sin depósito**. Sabemos que: P(cancela) = 0,4; entre las que cancelan, 80 % eran flexibles y 90 % sin depósito; entre las que no cancelan, 40 % flexibles y 50 % sin depósito.

```
 Puntuación "cancela"    = 0,4 · 0,8 · 0,9 = 0,288
Puntuación "no cancela" = 0,6 · 0,4 · 0,5 = 0,120

P(cancela | flexible, sin depósito) = 0,288 / (0,288 + 0,120) ≈ 0,706  → 71 %
```

- **Muy rápido y con pocos datos.**
- La suposición de independencia **casi nunca es cierta** (flexible y sin depósito suelen ir juntas), **pero a menudo funciona**.
- Hay dos versiones: `GaussianNB` (números) y `CategoricalNB` (categorías; los números se agrupan en «cajones» con `KBinsDiscretizer`).

14. ¿Qué modelo para el equipo de reservas?

Modelo	Ventaja	Pega
Regresión logística	**Explicable** (odds ratio), rápida	Solo fronteras lineales
k-NN	Sin supuestos, simple	**Lento al predecir**; necesita escalado
Naive Bayes	Muy rápido, pocos datos	Independencia poco realista

> **El mejor modelo no es solo el de mayor AUC:** cuentan el **tiempo** (`time.perf_counter()` para medirlo) y la **explicabilidad**. Un modelo con AUC 0,80 que el gerente entiende y corre en un segundo puede valer más que uno con 0,82 que nadie explica.

15. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	Practica en
Coeficientes, residuos, interacciones, polinomios	1-3	`S07_01_regresion_lineal_polinomica`
Sobreajuste, Ridge, Lasso	4-5	`S07_02_ridge_lasso`
Sigmoide, odds ratio, umbral y coste	6-9	`S07_03_regresion_logistica`
Desbalance, ROC, PR, remedios	10-11	`S07_04_roc_pr_desbalance`
k-NN, Naive Bayes, comparar modelos	12-14	`S07_05_knn_naive_bayes`
Matriz de confusión paso a paso	8	Ejercicio HTML de matriz de confusión (`actividades/`)
Todo junto	todas	Reto: tarifa + riesgo de cancelación + informe

16. Chuleta de vocabulario

Palabra	En lengua normal
Regresión	Predecir un número
Coeficiente	Cuánto cambia la predicción al subir una variable en 1
Residuo	Real − predicho
Interacción	El efecto de una variable depende de otra
Feature engineering	Inventar variables útiles a partir de los datos
Regularización	Multa a los coeficientes grandes para evitar el sobreajuste

Palabra	En lengua normal
Ridge / Lasso	Encoge todos / lleva algunos a cero
alpha	Fuerza de la multa
Sigmoide	Función que convierte cualquier número en probabilidad
Odds	Prob. de ocurrir / prob. de no ocurrir
Odds ratio	Por cuánto se multiplican los odds
Umbral	Probabilidad desde la que se da el aviso
FN / FP	Cancelación no prevista / falsa alarma
Desbalance	Una clase es mucho más rara que la otra
ROC / AUC	Curva y área que miden cómo ordena el modelo
PR / AP	Curva y área de precisión frente a recall
Prevalencia	Proporción de positivos en los datos
k-NN	Predecir mirando a los k vecinos más cercanos
Naive Bayes	Multiplicar probabilidades suponiendo independencia

17. Cómo estudiar estos apuntes

1. **Antes de cada cuaderno**, lee la sección que indica la tabla del apartado 15.
2. **Haz a mano el ejemplo del umbral** (sección 9): es la pieza central del reto y lo que debes saber explicar con tus palabras.
3. **Para cada modelo**, ten claro: *¿qué predice? ¿cómo se interpreta? ¿cuándo falla?* (La tabla de la sección 14 te lo resume.)
4. **En el informe al cliente, traduce a euros**: «recall 75 %» no dice nada a un gerente; «evitamos 3 de cada 4 habitaciones vacías» sí.